

Machine Learning Notes

Chapter 1: Preliminary Introduction to Deep Learning

Vitor N. Cabral de Oliveira

vnco@cin.ufpe.br

August 18, 2026

This notebook serves as a comprehensive repository for my Machine Learning studies, with a primary focus on Deep Learning. In this chapter, we establish the foundational concept of mathematical modeling, explore the biological inspiration behind Artificial Neural Networks (ANNs), and formalize their mathematical structure as parametric function approximators.

Contents

<i>The Concept of Modeling</i>	3
<i>The Learning Paradigm</i>	3
<i>The Biological Inspiration</i>	4
<i>Anatomy of a Neuron</i>	4
<i>The Action Potential and Non-Linearity</i>	5
<i>From Biology to Mathematics: The Artificial Neuron</i>	5
<i>Activation Functions</i>	7
<i>Multilayer Perceptron (MLP)</i>	7
<i>Measuring the Error: Loss Functions</i>	8
<i>Mean Squared Error (MSE)</i>	8
<i>Cross-Entropy Loss</i>	9
<i>Universal Approximation Theorem</i>	9

The Concept of Modeling

Before diving into neurons and weights, we must understand what it means to *model* something. In the physical sciences, a model is a simplified representation of a complex reality. A map is a model of a city; it discards irrelevant details (the color of individual houses) to highlight essential structures (streets and distances).

In machine learning, we take a formal approach. We assume there is a true, underlying relationship between some input variables $\mathbf{x}$ and an output y . We cannot observe this relationship directly, but we can observe a dataset $\mathcal{D} = \{(\mathbf{x}_d, y_d)\}_{d=1}^N$.

Definition 0.1 (Mathematical Model) A mathematical model is a parametric function $f_\theta : \mathcal{X} \rightarrow \mathcal{Y}$ that maps an input space $\mathcal{X}$ to an output space $\mathcal{Y}$. The vector $\theta \in \Theta$ represents the **parameters** of the model, drawn from a hypothesis space Θ .

The Learning Paradigm

In traditional programming, a human engineer writes explicit rules (code) to transform data into answers. In machine learning, we invert this process: we provide the data and the answers, and the algorithm searches for the rules.

To "learn" means to find the specific parameters θ^* that make the model's predictions $f_\theta(\mathbf{x})$ closely match the true outputs y for unseen data. This requires three components:

1. **The Hypothesis Space:** The set of all functions our model can represent. A linear model can only draw straight lines; a neural network can draw highly complex curves.
2. **The Loss Function:** A metric $\mathcal{L}(\hat{y}, y)$ that quantifies the penalty for an incorrect prediction.
3. **The Optimization Algorithm:** A procedure to navigate the hypothesis space and find θ^* that minimizes the loss.

Remark 0.1 (Inductive Bias) No model can learn without assumptions. If we assume the relationship between $\mathbf{x}$ and y is linear, our hypothesis space is small and easy to optimize, but it will fail if the true relationship is complex. If we use a highly flexible model (like a deep neural network), our hypothesis space is vast, but we need vast amounts of data to find the correct parameters without merely memorizing the training set. The assumptions we embed into the model's architecture are called its *inductive bias*.

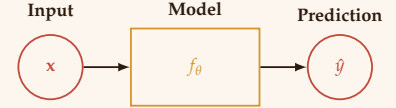


Figure 1: The standard ML pipeline: a model f_θ maps inputs to outputs.

The Biological Inspiration

To understand why artificial neural networks are structured the way they are, we must first look at their biological counterpart: the human brain. The brain is the most powerful learning machine known, capable of recognizing patterns, understanding language, and making complex decisions, all while consuming roughly 20 watts of power.

The fundamental unit of this biological processor is the **neuron**. The human brain contains approximately 86 billion neurons, connected in vast networks. To build our mathematical model, we will abstract the key functional mechanisms of these biological cells.

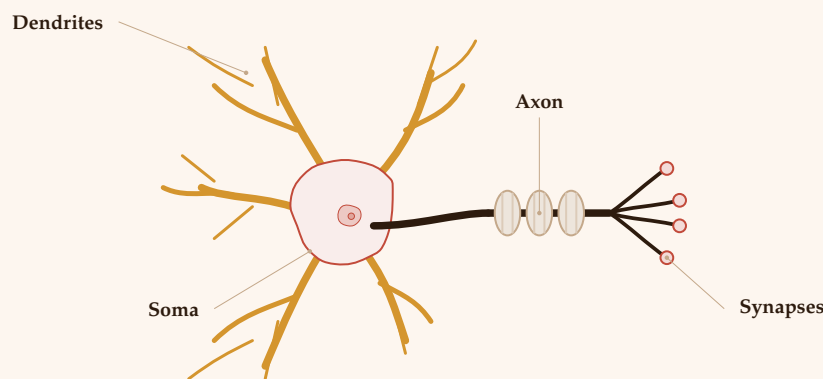


Figure 2: Anatomy of a biological neuron. Electrical signals flow from dendrites, through the soma, down the axon, to the synapses.

Anatomy of a Neuron

A biological neuron is a specialized cell that processes and transmits electrical and chemical signals. It consists of three main structural components:

1. **Dendrites:** These are tree-like extensions that act as the "antennae" of the neuron. They receive signals (neurotransmitters) from other neurons. The physical structure of the dendrites determines how the neuron integrates these incoming signals.
2. **Soma (Cell Body):** The soma contains the nucleus of the cell. Its critical function is to aggregate the electrical signals received from the dendrites. It acts as an integrator, summing up the excitatory and inhibitory inputs.
3. **Axon:** If the aggregated signal in the soma exceeds a certain threshold, the neuron "fires." This generates an electrical impulse called an action potential, which travels down the axon. The axon is the output wire of the neuron.

4. **Synapses:** At the end of the axon are terminal branches that form connections (synapses) with the dendrites of other neurons. The *strength* of these synaptic connections determines how much influence one neuron has on another.

The Action Potential and Non-Linearity

A crucial biological feature is how the neuron decides to fire. The soma does not simply pass along the sum of its inputs linearly. Instead, it operates on an "all-or-nothing" principle.

When the aggregated electrical potential inside the cell reaches a specific *threshold*, voltage-gated ion channels open, causing a massive, rapid spike in voltage—the action potential. If the input is below the threshold, nothing happens; the neuron remains silent.

Furthermore, the strength of the connections (synapses) is not fixed. Biological synapses exhibit **plasticity**. If two neurons fire together frequently, the synaptic connection between them strengthens. This is known as Hebbian theory: "Neurons that fire together, wire together." This biological adaptation of connection strengths is the direct inspiration for how artificial neural networks "learn" by adjusting weights.

From Biology to Mathematics: The Artificial Neuron

In the 1940s and 1950s, researchers like Warren McCulloch and Walter Pitts sought to abstract the biological neuron into a mathematical model. They recognized that the structural components of a biological neuron could be translated into algebraic operations.

Biological Concept	Mathematical Abstraction	Symbol
Dendrites (receiving signals)	Input vector	$\mathbf{x}$
Synaptic strength	Weight parameter	$\mathbf{w}$
Soma (signal integration)	Dot product (weighted sum)	$\mathbf{w} \cdot \mathbf{x}$
Baseline firing threshold	Bias parameter	b
Action Potential (all-or-nothing)	Non-linear activation function	$\phi(\cdot)$

Table 1: Mapping biological mechanisms to the mathematical components of an artificial neuron.

A single artificial neuron (historically called a Perceptron) is fundamentally an affine transformation followed by a non-linear function. It maps an input vector $\mathbf{x} = (x_1, \dots, x_d)^T \in \mathbb{R}^d$ to an output o :

$$o = \phi(\mathbf{w} \cdot \mathbf{x} + b)$$

Here, the parameters θ are the weights $\mathbf{w} \in \mathbb{R}^d$ and the bias $b \in \mathbb{R}$. But what does the operation $\mathbf{w} \cdot \mathbf{x}$ actually represent?

Geometrically, the dot product is a measure of *alignment* between two vectors. If the input vector x points in a similar direction to the weight vector w , their dot product is a large positive number. If they point in opposite directions, it is negative. If they are perpendicular, it is zero. Therefore, the weights act as a *filter*: the neuron outputs a high value only when the input matches a specific pattern encoded in w .

The bias b acts as a baseline threshold. It shifts the activation point, allowing the neuron to fire even when the alignment with w is not perfectly positive, or requiring a very strong alignment to fire at all.

Geometrically, the weights define a hyperplane that cuts the space. A single neuron can only represent *linearly separable* functions; the XOR problem is the classic counterexample where no single hyperplane can partition the activation states.

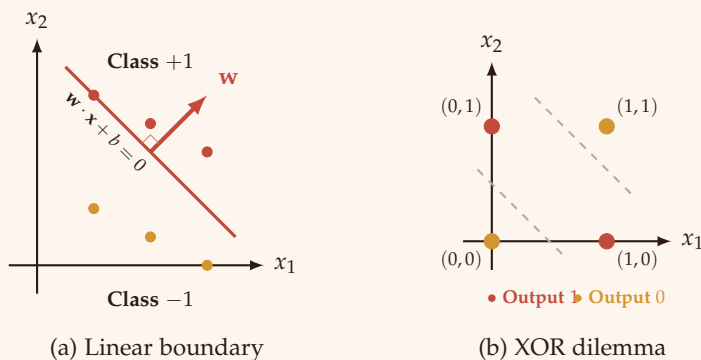


Figure 3: Geometric representation of linear separability. (a) A single hyperplane partitions space based on the weight vector w . (b) The XOR problem cannot be solved by a single linear hyperplane.

Listing 1: Implementing the Neuron class in `mini_nn/primitives.py`.

```

1 import numpy as np
2
3 class Neuron:
4     def __init__(self, in_dim):
5         # Random initialization for the forward pass
6         self.w = np.random.randn(in_dim)
7         self.b = np.random.randn()
8
9     def __call__(self, x):
10        # Affine transformation:  $w \cdot x + b$ 
11        z = np.dot(self.w, x) + self.b
12        # Non-linear activation (ReLU)
13        return np.maximum(0, z)

```

Activation Functions

For multilayer networks, we need nonlinear, differentiable activation functions. Without them, stacking layers would merely result in a single linear transformation ($\mathbf{W}_2(\mathbf{W}_1\mathbf{x}) = \mathbf{W}'\mathbf{x}$).

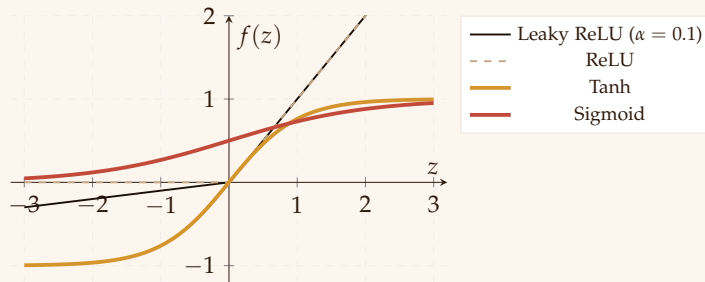


Figure 4: Common activation functions. Sigmoid and tanh saturate at the tails, causing gradients to vanish. ReLU is piecewise linear, preserving gradients for positive inputs.

Geometrically, linear transformations ($\mathbf{W}\mathbf{x}$) only stretch, rotate, or shear the space. Activation functions are responsible for *bending* or *folding* the space. The Rectified Linear Unit (ReLU), defined as $\max(0, z)$, acts like origami: it folds the negative quadrants of the space onto the boundaries of the positive ones, allowing the network to carve complex decision boundaries out of the input space.

Multilayer Perceptron (MLP)

To solve non-linear problems like XOR, we stack multiple layers of neurons. A multilayer perceptron (MLP) consists of an input layer, one or more hidden layers, and an output layer.

Instead of computing neurons one by one, we use linear algebra to compute an entire layer simultaneously. Let $\mathbf{o}^{(\ell-1)} \in \mathbb{R}^{d_{in}}$ be the output of the previous layer. We define a weight matrix $\mathbf{W}^{(\ell)} \in \mathbb{R}^{d_{out} \times d_{in}}$ and a bias vector $\mathbf{b}^{(\ell)} \in \mathbb{R}^{d_{out}}$. The forward pass for layer ℓ is:

$$\mathbf{a}^{(\ell)} = \mathbf{W}^{(\ell)} \mathbf{o}^{(\ell-1)} + \mathbf{b}^{(\ell)}, \quad \mathbf{o}^{(\ell)} = f^{(\ell)}(\mathbf{a}^{(\ell)}),$$

where each *row* of $\mathbf{W}^{(\ell)}$ corresponds to the weights of a single neuron in the new layer. The matrix multiplication efficiently computes the dot products of all neurons against the input simultaneously.

Listing 2: Implementing the Layer and MLP classes in `mini_nn/layers.py`.

```
1 class Layer:
2     def __init__(self, in_dim, out_dim):
3         self.W = np.random.randn(out_dim, in_dim)
4         self.b = np.random.randn(out_dim)
5
6     def __call__(self, x):
```

```

7      # Affine transformation for a single input vector x (a
      # column of a batch)
8      z = np.dot(self.W, x) + self.b
9      return np.maximum(0, z) # ReLU activation
10
11 class MLP:
12     def __init__(self, sizes):
13         # sizes: [in_dim, hidden_1, hidden_2, ..., out_dim]
14         self.layers = [Layer(sizes[i], sizes[i+1])
15                        for i in range(len(sizes)-1)]
16
17     def __call__(self, x):
18         for layer in self.layers:
19             x = layer(x) # Forward pass sequentially
20         return x

```

Remark 0.2 (Bridge to Chapter 2) *This chapter implements the forward pass using plain numpy: the weights $\mathbf{W}$ are static arrays, and the network cannot improve its predictions. In Chapter 2 we will replace these arrays with *Valor*—a tiny automatic-differentiation engine—so that each weight becomes a differentiable node of a computational graph. The forward computation stays exactly the same (affine transformation followed by an activation); the only difference is that every operation now records how it was computed, enabling the backward pass that makes the network trainable. The *mini_nn* framework is thus upgraded in place: same architecture, differentiable core.*

Measuring the Error: Loss Functions

Once the network produces an output $\hat{\mathbf{y}} = \mathbf{o}^{(L)}$, we need a mechanism to measure how far it is from the target $\mathbf{t}$. This is the role of the **Loss Function** $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{t})$. The choice of loss function dictates how the network penalizes mistakes.

Mean Squared Error (MSE)

For regression tasks (predicting continuous numbers), we want to penalize the distance between the prediction and the target. We use the squared error:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{2} \|\mathbf{t} - \hat{\mathbf{y}}\|^2$$

Why square it? First, it ensures the error is always positive. Second, it heavily penalizes large errors (being off by 10 costs 100, while being off by 1 costs 1), which encourages the network to fix its worst mistakes first.

Cross-Entropy Loss

For classification tasks, the network outputs raw scores called *logits*. We first use the Softmax function to squash these into probabilities that sum to 1:

$$\text{Softmax}(\hat{y}_i) = \frac{e^{\hat{y}_i}}{\sum_j e^{\hat{y}_j}}$$

To measure the error, we use Cross-Entropy. Unlike MSE, Cross-Entropy punishes the network exponentially if it is *confident but wrong*. If the true class is i , the loss is:

$$\mathcal{L}_{\text{CE}} = -\log(\text{Softmax}(\hat{y}_i))$$

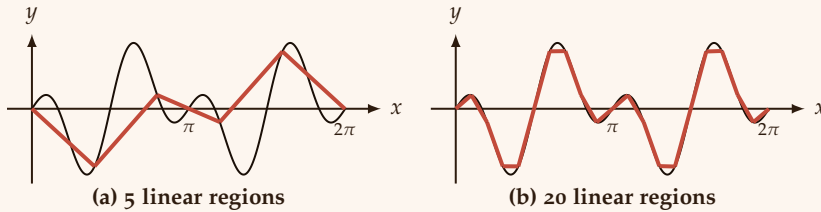
If the network predicts a probability of 0.99 for the true class, $\mathcal{L} \approx 0.01$. If it predicts 0.01, $\mathcal{L} \approx 4.6$. This steep penalty forces the network to learn the correct class boundaries quickly.

Universal Approximation Theorem

Theorem 0.1 (Cybenko, 1989) Let $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ be a continuous, bounded, non-constant activation function. Then for any continuous function $f : [0, 1]^d \rightarrow \mathbb{R}$ and any $\varepsilon > 0$, there exist $N \in \mathbb{N}$, vectors $\mathbf{w}_j \in \mathbb{R}^d$, scalars $v_j, b_j \in \mathbb{R}$ such that

$$\sup_{\mathbf{x} \in [0, 1]^d} \left| \sum_{j=1}^N v_j \sigma(\mathbf{w}_j \cdot \mathbf{x} + b_j) - f(\mathbf{x}) \right| < \varepsilon.$$

The theorem guarantees that the hypothesis space of an MLP (with enough neurons) contains a function that can approximate any continuous function arbitrarily well. This proves that our architectural components possess the mathematical capacity to model complex phenomena. However, the theorem says nothing about *how* to find those parameters θ .



Remark 0.3 We have defined our hypothesis space (f_θ) and our objective ($\mathcal{L}$). However, our `mini_nn` model currently relies on random initialization; it has no mechanism to improve its predictions. In Chapter 2, we will solve this by introducing Differentiable Programming and the optimization algorithms used to find θ^* .

Figure 5: Visualization of the Universal Approximation Theorem. The dark curve is a highly oscillatory target function $f(x) = 1.5 \sin(x) \cos(3x)$. The terracotta lines represent the piecewise linear approximation formed by a neural network. With only 5 regions (a), the network misses the internal fluctuations entirely. With 20 regions (b), the approximation becomes highly precise.